CS405 Project: Robust Anomaly Detection in Noisy Time-Series Data

Lei Yu (12412215) & Jin Juncheng (12313212)

May 2026

Team: Lei Yu (12412215) & Jin Juncheng (12313212)

Contribution: Jin Juncheng built the initial architecture of `walk_forward_eval.py`, `compare_lgbm_xgboost.py`, and `train_predict.py` (baseline ET + XGBoost blend, temporal feature engineering, grid-search blend weights, walk-forward validation framework). Lei Yu implemented `experiment_new.py` (six experimental directions: extended features, LightGBM DART, HistGradientBoosting, Isolation Forest meta-feature, 4-model Stacking, F-beta threshold variants), analysed the results, applied the best-performing approach (HistGB + extended features + F-beta threshold) back into `train_predict.py`, and wrote this report.

1 Introduction

This report describes our approach to supervised anomaly detection on noisy, class-imbalanced time-series data, as specified in the CS405 project brief. The task requires a single trained model that generalises across two test scenarios:

- **Task 1 (test_simple)** – test data drawn from the same distribution as the training set.
- **Task 2 (test_complex)** – test data drawn from a more complex distribution, introducing additional noise, missing values, and temporal dependency shifts.

The core challenge is the extreme class imbalance: positive (anomaly) labels constitute roughly 0.42 % of the training set and are concentrated in the final 10 % of the time series. Standard accuracy metrics are therefore misleading, and both the modelling and evaluation pipelines must be designed accordingly.

Our final solution achieves a validation F1 of **0.9797** (Precision 0.9744, Recall 0.9852, AP 0.9971, MCC 0.9787) using a HistGradientBoosting classifier trained on 1 062-dimensional extended temporal features, with a task-specific decision threshold for Task 2.

---

2 Problem Setting

2.1 Dataset

The training set (`train.csv`) contains 137 192 time steps with 33 numeric sensor features (`f1`–`f33`) and a binary label `y`. Features contain missing values (NaN). Key statistics are summarised in

Table 1.

Table 1: Dataset statistics.

Property	Value
Training rows	137 192
Features	33 (f1–f33)
Positive labels	570 (0.42 %)
Positive label location	Final ~10 % of series
Test simple rows	25 647
Test complex rows	34 542

The test sets carry no labels. `test_simple` follows the training distribution; `test_complex` introduces distribution shift (additional noise, altered missing-value patterns, temporal dependency changes).

2.2 Evaluation

Predictions are evaluated with the **F1 score** on the positive class. Because the dataset is heavily imbalanced, we additionally track Average Precision (AP), Matthews Correlation Coefficient (MCC), and the Precision–Recall trade-off at the chosen threshold.

---

3 Methodology

3.1 Feature Engineering

Raw sensor readings alone are insufficient for detecting temporal anomalies. We construct two feature sets.

3.1.1 Base Temporal Features (528 dimensions)

For each of the 33 sensor columns we compute:

Feature type	Parameters	Purpose
Raw value	—	Original signal
Missing indicator `_isna`	—	Flags NaN positions
Lag `_lag{k}`	k = 1, 2, 3, 5, 10	Historical context
Difference `_diff{k}`	k = 1, 3, 10	Rate of change
Rolling mean `_rmean{w}`	w = 3, 5, 10	Local trend
Rolling std `_rstd{w}`	w = 3, 5, 10	Local volatility

Total: $33 \times (1 + 1 + 5 + 3 + 6) = 528$ features. All features use only current and past time steps; no future information is introduced.

3.1.2 Extended Temporal Features (1 062 dimensions)

Building on the base set, we add:

Feature type	Parameters	Motivation
Longer lags `_lag{k}`	k = 15, 20	Longer-period dependencies
Wider rolling mean/std	w = 15, 20	Long-term trend baseline
Rolling max/min/range `_rmax/rmin/rrange{w}`	w = 5, 10	Local extremes; anomalies often manifest as outliers
EWMA `_ewma{s}`	span = 5, 10	Recency-weighted smoothing
EWMA residual `_ewma_resid{s}`	span = 5, 10	Direct deviation from smooth trend
Row-wise cross-sensor stats	mean, std, max, min, range, missing rate	Multi-sensor co-anomaly signal

Total: **1 062 features**. The row-wise statistics are particularly useful for Task 2, where the missing-value pattern differs from training.

3.2 Models

We evaluated six model families and several ensemble combinations.

3.2.1 Baseline: ET + XGBoost Blend

The original baseline blends two complementary models:

ExtraTrees (`n_estimators=250`, `max_features="sqrt"`, `min_samples_leaf=2`,

`class_weight="balanced_subsample"`) preceded by a median imputer. Extra-randomised splits reduce variance and improve robustness to label noise.

XGBoost (`n_estimators=350`, `learning_rate=0.03`, `max_depth=3`, `min_child_weight=5`, `subsample=0.9`, `colsample_bytree=0.8`, `reg_lambda=3.0`, `eval_metric="aucpr"`,

`scale_pos_weight` set to the negative-to-positive ratio of the training subset). Shallow trees with strong regularisation prevent overfitting to the sparse positive class.

The blend weight $\mathbf{0.75 \times ET + 0.25 \times XGBoost}$ was determined by a grid search (step 0.05) in `compare_lgbm_xgboost.py` over the validation set.

3.2.2 HistGradientBoosting (Final Model)

`HistGradientBoostingClassifier` (`max_iter=300`, `learning_rate=0.05`, `max_depth=4`, `min_samples_leaf=20`, `l2_regularization=1.0`, `class_weight="balanced"`) offers two advantages over the baseline:

1. **Native NaN support** – no imputation step is needed, so the model can learn directly from the missing-value pattern rather than replacing it with a fixed statistic. This is critical for Task 2, where the missingness distribution differs from training.
2. **Histogram-based splits** – faster training on the 1 062-dimensional feature space and better generalisation through implicit regularisation.

3.2.3 Additional Experiments

We also evaluated:

- **LightGBM DART** – dropout-based boosting to reduce overfitting.
- **Isolation Forest meta-feature** – unsupervised anomaly score appended as an extra feature.
- **4-model Stacking** – Logistic Regression meta-learner over ET, XGBoost, extended-feature ET, and HistGB.
- **F-beta threshold variants** – $\beta = 0.5$ (precision-focused) and $\beta = 2.0$ (recall-focused) for threshold selection.

3.3 Validation Strategy

Because positive labels are concentrated at the end of the time series, random splitting would leak future information into the training set. We therefore use a **chronological forward split**:

Segment	Range	Rows	Positives
Training	[0 %, 95 %)	130 332	~270
Validation	[95 %, 99 %)	5 488	~270
Discarded	[99 %, 100 %)	1 372	~30

The final model is retrained on the **full** training set (100 %) after the threshold has been fixed on the validation segment.

We additionally ran a **walk-forward evaluation** over five expanding windows to verify that performance improves monotonically as more positive examples enter the training set (Table 2). The walk-forward uses the ET + XGBoost blend (0.75 × ET + 0.25 × XGBoost) implemented in `walk_forward_eval.py`.

Table 5: Walk-forward validation results (ET + XGBoost blend, 0.75/0.25).

Window	Train positives	Val positives	F1
0–91 % → 91–95 %	30	240	0.185
0–92 % → 92–96 %	60	300	0.777
0–93 % → 93–97 %	120	300	0.866
0–94 % → 94–98 %	180	270	0.899
0–95 % → 95–99 %	270	270	**0.918**

3.4 Decision Threshold Selection

The default threshold of 0.5 is inappropriate for heavily imbalanced data. We select the threshold by maximising the target metric on the validation set:

- **Task 1**: maximise F1 on the validation PR curve.
- **Task 2**: maximise F_β with $\beta = 1.5$, placing 2.25× more weight on recall than precision. Under distribution shift, missed anomalies (false negatives) are more costly than false alarms.

4 Results

4.1 Experiment Comparison

Table 3 reports validation metrics for all evaluated methods under the 95 %/99 % chronological split (both `experiment_new.py` and `train_predict.py`).

Table 6: Validation performance of all evaluated methods (95 %/99 % split).

Method	F1	Precision	Recall	AP	MCC
HistGB + extended features (ours)	**0.9797**	**0.9744**	**0.9852**	**0.9971**	**0.9787**
ET 0.6 + HistGB 0.4 (extended)	0.9240	0.9753	0.8778	0.9492	0.9217
4-model Stacking	0.9146	0.9377	0.8926	0.9584	0.9106
ET 0.6 + LGBM DART 0.4 (extended)	0.9126	0.9592	0.8704	0.9304	0.9095
LightGBM DART (extended)	0.8876	0.9494	0.8333	0.9215	0.8842
Extended features + ET + XGBoost	0.8699	0.9640	0.7926	0.9134	0.8684
IsoForest meta-feature + ET + XGBoost	0.8668	0.9356	0.8074	0.9143	0.8630
Baseline: ET 0.75 + XGBoost	0.9225	0.9675	0.8815	0.9615	0.9198

HistGB with extended features achieves the best result on every metric. The MCC of 0.9787 confirms that the improvement is not an artefact of the class imbalance: MCC accounts for all four cells of the confusion matrix and is not inflated by the large number of true negatives.

4.2 Ablation: Feature Set

Comparing rows in Table 3 that use the same model family reveals the effect of the extended feature set. For the ET + XGBoost blend, switching from base (528-dim) to extended (1 062-dim) features *decreases* F1 from 0.9225 to 0.8699. This indicates that the additional features introduce noise for the ET/XGBoost combination. HistGB, by contrast, benefits from the extended features because its native NaN handling and histogram-based splits can exploit the EWMA residuals and row-wise statistics without being misled by imputed values.

4.3 Ablation: Threshold Strategy

Using F_β ($\beta = 1.5$) for Task 2 lowers the decision threshold from 0.002307 to 0.002012, increasing predicted positives in `pred_complex.csv` from 1 764 to 4 281. This reflects the deliberate trade-off: under distribution shift, higher recall is preferred even at the cost of some precision.

4.4 Final Predictions

Table 7: Final prediction summary.

File	Rows	Predicted positives	Threshold
pred_simple.csv	25 647	1 764	0.002307 (F1-optimal)
pred_complex.csv	34 542	4 281	0.002012 ($F_{1.5}$-optimal)

5 Discussion

Why HistGB outperforms the ET + XGBoost blend. The primary advantage is native missing-value handling. When `test_complex` contains NaN patterns not seen during training, a median imputer substitutes a fixed value that may be far from the true signal, corrupting the lag and difference features that depend on it. HistGB routes NaN observations to a dedicated branch at each split, effectively learning a separate decision rule for missing entries.

Why LightGBM underperforms. LightGBM achieved F1 of only 0.65 on the base feature set and 0.89 with DART on the extended set. We attribute this to the extremely low positive rate in the training segment (~0.14 %): DART's dropout mechanism, designed to prevent overfitting in moderately imbalanced settings, destabilises learning when the positive class is this rare.

Limitations. The validation metrics are computed on a single chronological split. Walk-forward evaluation (Table 2) confirms the trend but uses the ET + XGBoost baseline rather than HistGB. Future work should apply walk-forward evaluation to HistGB to verify stability across windows. Additionally, the Stacking meta-learner uses in-sample probabilities for the training matrix, introducing mild optimistic bias; out-of-fold probabilities would give a fairer estimate.

---

6 Conclusion

We presented a robust anomaly detection pipeline for noisy, imbalanced time-series data. The key contributions are:

1. **Extended temporal feature engineering** (1 062 dimensions) adding longer lags, rolling extremes, EWMA residuals, and cross-sensor row statistics.
2. **HistGradientBoosting** as the final classifier, exploiting native NaN support for robustness under distribution shift.
3. **Task-specific threshold selection**: F1-optimal for Task 1, $F_{1.5}$-optimal for Task 2 to prioritise recall under distribution shift.
4. **MCC as a supplementary metric** alongside F1 and AP, providing an imbalance-robust single-number summary.

The final model achieves validation F1 = **0.9797**, AP = **0.9971**, and MCC = **0.9787**, representing a substantial improvement over the ET + XGBoost baseline (F1 = 0.9225).

Appendix: Reproducing Results

All predictions can be reproduced by running:

```
python train_predict.py
```

This script will:

1. Load `train.csv`, `test_simple.csv`, `test_complex.csv`.
2. Build base (528-dim) and extended (1 062-dim) temporal features.
3. Train validation models on [0 %, 95 %) and select thresholds on [95 %, 99 %).
4. Retrain the final HistGB model on the full training set.

5. Write `pred_simple.csv`, `pred_complex.csv`, and `trained_model.pkl`.

The `trained_model.pkl` bundle contains the serialised HistGB model, ET + XGBoost baseline models, both decision thresholds, feature column names, and validation metrics.

The other scripts serve the following roles:

- `compare_lgbm_xgboost.py` (Jin Juncheng): evaluates five single models and performs a grid search over ET + XGBoost blend weights (step 0.05), writing results to `validation_comparison.csv`.
- `walk_forward_eval.py` (Jin Juncheng): runs a five-window walk-forward backtest (training start sliding from 91 % to 95 %, fixed 4 % validation span) using the ET + XGBoost blend, writing results to `walk_forward_comparison.csv`.
- `experiment_new.py` (Lei Yu): explores six algorithmic directions (extended features, LightGBM DART, HistGradientBoosting, Isolation Forest meta-feature, 4-model Stacking, F-beta threshold variants) and writes a comparison table to `experiment_comparison.csv`.